

Customer Churn Prediction Using Data Mining Technique

Course Project Documentation

Documentation structure

1. Problem Description
2. Dataset Description
3. Data Preprocessing
4. Data Mining Techniques & Algorithms
5. Implementation Tool
6. Results & Discussion
7. Conclusion
8. Future Work
9. GitHub Repository

1. Problem Description

Customer churn occurs when a customer stops using a company's service or ends their relationship with the company.

The problem addressed in this project is **predicting whether a customer is likely to churn** based on their demographic, behavioral, and subscription-related information.

The project applies **Data Mining classification techniques** to learn patterns from historical customer data and use these patterns to classify customers into two classes:

- **Churn (1)** — the customer is expected to churn.
- **Stay (0)** — the customer is expected to remain with the company.

The project also compares different classification algorithms to determine which approach performs best on the given dataset.

Project Objectives

The main objectives are to:

1. Prepare and preprocess the customer dataset.

2. Apply classification algorithms to the churn prediction problem.
3. Evaluate the performance of the selected algorithms.
4. Compare the obtained results.
5. Investigate patterns in the data and learned decision rules.
6. Select the best-performing model for predicting churn for new customers.

2. Dataset Description

The project uses the **Customer Churn Dataset** obtained from Kaggle.

Source: [Customer Churn Dataset — Kaggle](#)

The dataset contains customer information related to demographics, customer behavior, usage, payments, and subscription characteristics.

2.1 Dataset Features

The following features are used as input variables:

- **Age** — customer's age.
- **Gender** — customer's gender.
- **Tenure** — duration of the customer's relationship with the service.
- **Usage Frequency** — frequency of service usage.
- **Support Calls** — number of support calls made by the customer.
- **Payment Delay** — payment delay associated with the customer.
- **Subscription Type** — customer's subscription type.
- **Contract Length** — customer's contract duration.
- **Total Spend** — customer's total spending.
- **Last Interaction** — number associated with the customer's most recent interaction.

2.2 Target Variable

The target variable is **Churn**, which represents whether the customer churned:

- **1** → Customer churned
- **0** → Customer did not churn

2.3 Identifier

The dataset also contains **CustomerID**.

`CustomerID` is an identifier rather than a predictive feature. It does not provide meaningful information for determining whether a customer will churn, so it was removed before training the classification models.

2.4 Dataset Size

The training dataset contains 440,882 customer records.

The project applies an 80/20 train-test split to this training data.

The resulting test portion contains 88,167 records.

After preprocessing, the test data contains 15 processed features.

3. Data Preprocessing

Before applying the classification algorithms, the dataset was cleaned and transformed into a format suitable for machine learning.

3.1 Handling Missing Values

Rows containing missing values were removed using `dropna()`.

This ensures that the models receive complete records during training and evaluation.

3.2 Removing the Customer Identifier

The `CustomerID` column was removed because it is an identifier and does not provide meaningful predictive information about customer churn.

The remaining columns were used as potential predictive features.

3.3 Separating Features and Target

The dataset was divided into:

- **Features (X)** — customer information used by the models.
- **Target (y)** — the `Churn` variable that the models are trained to predict.

Customer Features (X)

↓

Classification Model



Churn (y)

3.4 Identifying Feature Types

The features were separated into categorical and numerical variables.

Categorical features:

- Gender
- Subscription Type
- Contract Length

Numerical features:

- Age
- Tenure
- Usage Frequency
- Support Calls
- Payment Delay
- Total Spend
- Last Interaction

3.5 Encoding Categorical Features

Machine learning models require numerical input for these features. Therefore, categorical variables were transformed using **One-Hot Encoding**.

For example, a categorical value such as:

Contract Length = Monthly

is represented as a numerical indicator such as:

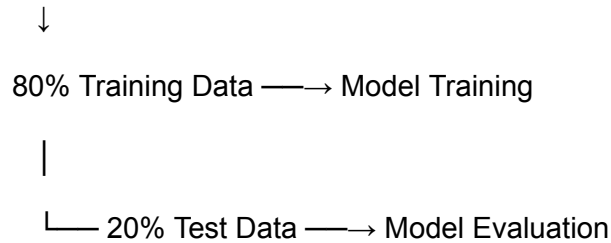
Contract Length_Monthly = 1

while other contract categories receive their corresponding encoded columns.

3.6 Train-Test Split

The data was divided into training and testing sets using an **80/20 split**.

Dataset



The training data is used to learn patterns, while the test data remains unseen during training and is used to evaluate the trained models.

3.7 Fitting and Applying the Preprocessor

The preprocessing transformer was fitted only on the training data:

```
X_train_processed = preprocessor.fit_transform(X_train)
```

The already-fitted transformer was then applied to the test data:

```
X_test_processed = preprocessor.transform(X_test)
```

This separation prevents information from the test data from influencing the preprocessing rules learned from the training data and helps avoid **data leakage**.

The same fitted preprocessing pipeline is also saved and reused when processing new customer information during prediction.

4. Data Mining Techniques and Algorithms

The project uses **classification**, a supervised Data Mining technique, because the target variable **Churn** contains two known classes: churn (1) and stay (0).

Three classification algorithms were selected and trained on the same preprocessed training data.

4.1 Logistic Regression

Logistic Regression was selected as a classical classification baseline for the binary churn prediction problem.

It provides a relatively simple model that can be used as a reference when comparing its performance with more complex classification approaches.

4.2 Decision Tree

Decision Tree was selected because it can learn decision rules based on customer features.

For example, the trained tree can learn rules involving thresholds such as:

Support Calls > 4.5



Support Calls > 5.5



Churn

This makes the Decision Tree useful not only for prediction but also for interpreting how the model reaches a classification decision.

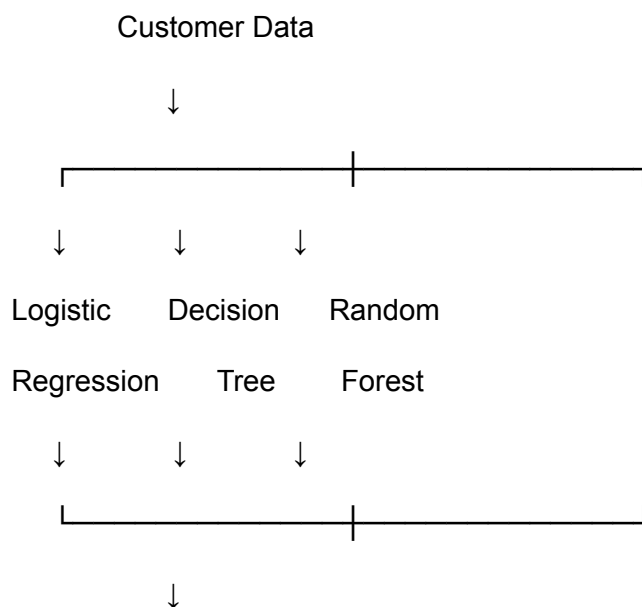
4.3 Random Forest

Random Forest was selected as an **ensemble classification method** based on multiple Decision Trees.

It provides a comparison between using a single Decision Tree and combining multiple trees to make predictions.

4.4 Algorithm Comparison

Using these three algorithms allows the project to compare different classification approaches on the same dataset:



Compare Results

The models were evaluated using the same test data and the same evaluation metrics to determine which algorithm performed best for this customer churn problem.

5. Implementation Tool

The project was implemented using **Python** as the main programming language.

The following tools and libraries were used:

- **Pandas** — loading and handling the dataset.
- **Scikit-learn** — preprocessing, classification algorithms, train-test splitting, and model evaluation.
- **Matplotlib** — visualization of results.
- **Seaborn** — visualization of confusion matrices and exploratory analysis.
- **Joblib** — saving and loading the trained model and preprocessing pipeline.
- **Streamlit** — providing a simple user interface for making new customer churn predictions.
- **uv** — Python project and dependency management.

The implementation was organized into separate modules for data exploration, preprocessing, evaluation, training, prediction, and the frontend.

6. Results and Discussion

6.1 Evaluation Metrics

The models were evaluated using:

- **Accuracy** — proportion of correctly classified customers.
- **Precision** — proportion of customers predicted as churn who actually churned.
- **Recall** — proportion of actual churn customers correctly identified.
- **F1-score** — balance between precision and recall.
- **Confusion Matrix** — shows correct and incorrect predictions for each class.

6.2 Model Results

Model	Accuracy	Recall	Precision	F1-Score
-------	----------	--------	-----------	----------

Logistic Regression	89.34%	88.54%	92.34%	90.40%
Decision Tree	99.99%	99.99%	99.99%	99.99%
Random Forest	99.94%	99.90%	99.99%	99.95%

The Decision Tree achieved the highest performance on the test set and was therefore selected as the final model.

6.3 Discussion of Findings

The three algorithms were trained using the same dataset and preprocessing procedure, but their results were noticeably different.

Logistic Regression achieved lower performance compared with the two tree-based models. The Decision Tree and Random Forest were able to capture stronger threshold-based and nonlinear relationships in the customer data.

The exploratory analysis also identified strong observed relationships between some features and churn. For example:

- Payment Delay values from 21 onward showed 100% observed churn.
- Monthly contracts showed 100% observed churn.

These patterns help explain why tree-based models can perform particularly well on this dataset, as Decision Trees can represent such relationships through explicit thresholds and branches.

However, the extremely high performance of the Decision Tree and Random Forest should be interpreted carefully. These results were obtained on this particular dataset and test split. High performance on one dataset does not necessarily guarantee the same performance on new real-world data.

6.4 Decision Tree Interpretation

The trained Decision Tree was inspected to understand the rules it learned during training.

For example, one prediction path can be represented as:

Support Calls > 4.5



Support Calls > 5.5



Churn

This illustrates how a Decision Tree makes a prediction by evaluating feature conditions from the root through successive branches until reaching a final class.

The tree therefore provides an interpretable representation of some of the patterns learned from the training data.

6.5 Final Model Selection

The Decision Tree was selected because it achieved the highest performance among the three evaluated models.

The selection was based on the overall evaluation results rather than using a single metric alone. The Decision Tree achieved approximately **99.99%** for accuracy, recall, precision, and F1-score on the test set.

The selected model was then saved for use in the final prediction system.

7. Prediction

After selecting the Decision Tree, the trained model and fitted preprocessing pipeline were saved using Joblib.

For a new customer, the prediction process is:

New Customer Information



Saved Preprocessor



Processed Features



Saved Decision Tree

↓

Prediction

↓

Stay / Churn

The preprocessing pipeline ensures that new customer information is transformed consistently with the data used during model training.

A prediction script was implemented to load the saved components and generate a churn prediction for a new customer.

8. Frontend

A small **Streamlit** interface was developed to provide an interactive way to use the final prediction system.

The user can enter customer information, including demographic, behavioral, and subscription-related attributes.

The entered information is passed through the saved preprocessing pipeline and then to the trained Decision Tree.

The interface displays the resulting prediction as either:

- **STAY**
- **CHURN**

This provides a simple user-facing demonstration of the completed classification system.

9. Project Structure

customer-churn/

|

|— data/

| |— customer_churn_dataset-training-master.csv/

| |— customer_churn_dataset-testing-master.csv/

|

```
├── models/
|   ├── decision_tree.pkl
|   └── preprocessor.pkl
|
├── src/
|   ├── explore_data.py
|   ├── preprocess.py
|   ├── evaluate.py
|   ├── train.py
|   ├── predict.py
|   └── app.py
|
├── README.md
├── pyproject.toml
└── uv.lock
```

10. Conclusion

This project demonstrates an end-to-end Data Mining classification workflow for customer churn prediction.

The project started with understanding and preparing the customer dataset, followed by preprocessing and classification using Logistic Regression, Decision Tree, and Random Forest.

The models were evaluated and compared using multiple performance metrics. The Decision Tree achieved the highest performance on the test set and was selected as the final model.

The project also investigated the relationships between customer features and churn and examined the decision rules learned by the Decision Tree. This provided a better understanding of not only **which model performed best**, but also **why different models produced different results**.

Finally, the selected model and preprocessing pipeline were saved and integrated into a simple Streamlit interface for predicting churn for new customers.

11. Future Work

Possible extensions of the project include:

- Evaluating the selected model on an independent dataset.
- Applying cross-validation to further assess model generalization.
- Investigating feature importance and additional model interpretation techniques.
- Applying hyperparameter tuning.
- Testing additional classification algorithms.
- Deploying the prediction system as a production application.

12. GitHub Repository

All project materials and source code are available in the GitHub repository:

Customer Churn Prediction:

<https://github.com/Salma22C/Customer-Churn-Predication>